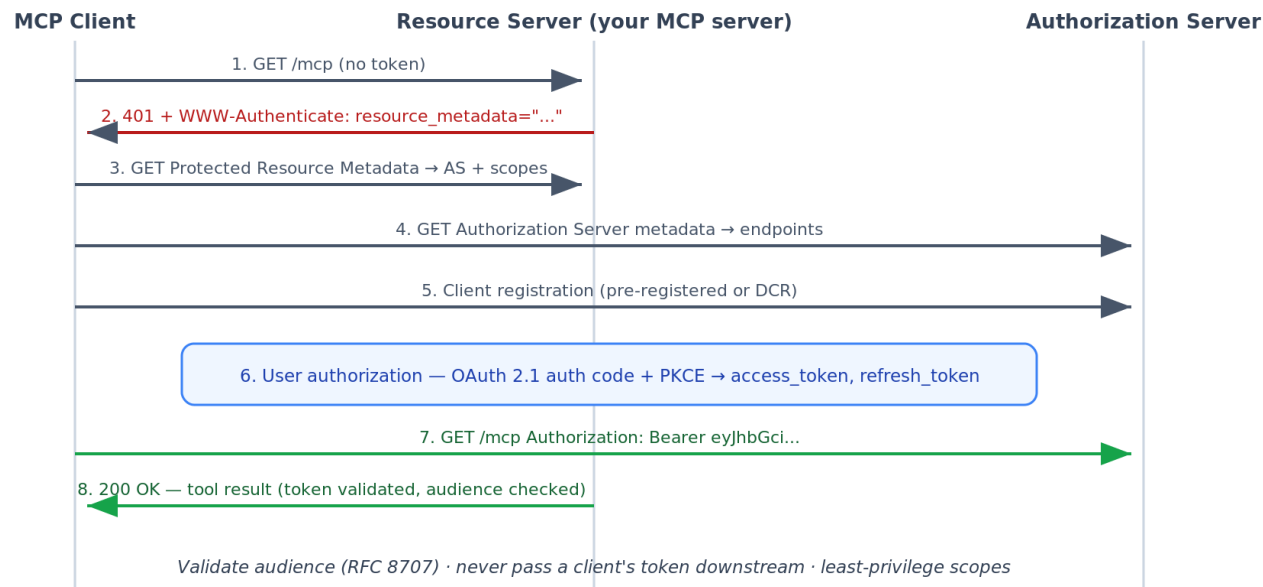


Section 1 – The OAuth 2.1 Resource-Server Model

In MCP, your server is a **Resource Server** - it holds the protected tools and resources. A separate **Authorization Server (AS)** issues tokens. The client discovers how to get a token and then presents it as a **Bearer** token.

OAuth 2.1 Resource-Server Model in MCP

The client discovers where to get a token, then presents it as a Bearer token



The discovery-to-token flow. The client learns where to authenticate, obtains a token, and retries the original request.

Step 1 – Submit request without Token.

The client has no idea yet that the server is protected, so it simply opens a session and sends its first MCP request with no Authorization header. This “fail first, then discover” pattern is deliberate: **the server, not the client, is the authority** on what authentication it requires.

HTTP – Unauthenticated request

```
POST /mcp HTTP/1.1
Host: your-server.com
Content-Type: application/json

{"jsonrpc": "2.0", "id": 1, "method": "initialize", ... }
```

Step 2 - The server challenges with 401 + where to find auth info

HTTP - 401 challenge response

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="mcp",
  resource_metadata="https://your-server.com/.well-known/oauth-protected-resource"
```

Step 3 - Protected Resource Metadata (PRM) discovery

The client fetches the pointer to learn the Authorization Server and the scopes it supports.

JSON - Protected Resource Metadata

```
{
  "resource": "https://your-server.com/mcp",
  "authorization_servers": ["https://auth.your-server.com"],
  "scopes_supported": ["mcp:tools", "mcp:resources"]
}
```

Step 4 - Authorization Server discovery

The client fetches the AS metadata (OIDC / OAuth 2.0 Authorization Server Metadata) for its endpoints.

JSON - Authorization Server metadata

```
{
  "issuer": "https://auth.your-server.com",
  "authorization_endpoint": "https://auth.your-server.com/authorize",
  "token_endpoint": "https://auth.your-server.com/token",
  "registration_endpoint": "https://auth.your-server.com/register"
}
```

Step 5 - Client registration

Either **pre-registered**, or **Dynamic Client Registration (DCR)** if the Authorization Server supports it.

Step 6 - User authorization

Standard **OAuth authorization code with PKCE**; the client exchanges the code for tokens (access_token, refresh_token, Bearer, expires_in).

Step 7 - Authenticated requests

The access token rides in the Authorization header:

HTTP - authenticated request

```
GET /mcp HTTP/1.1
Host: https://www.your-mcp-server.com
```

Authorization: Bearer eyJhbGciOiJSUzI1NiIs...

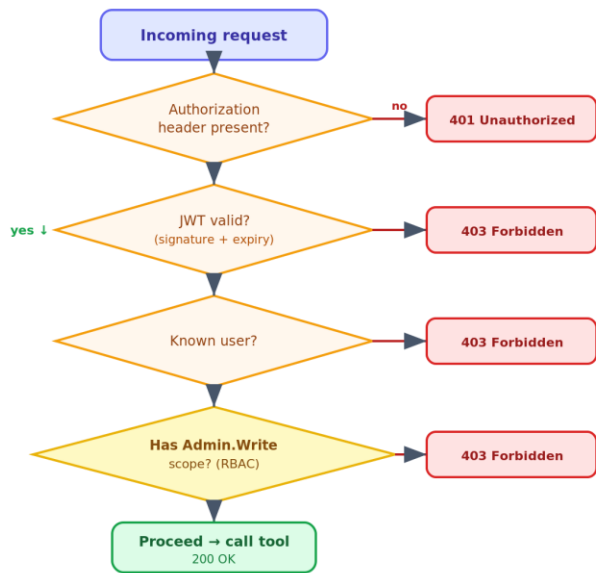
Section 2 - The Lab's Simplification (read this first)

This lab keeps the **resource-server enforcement** - the part you own - and simplifies token issuance to a **shared-secret HS256 JWT** minted locally.

Production	Lab
External Auth Server	util.py creates token
RS256	HS256
JWKS key rotation	Shared secret
Full OAuth validation	Simple JWT validation

The middleware enforces, on every request, a ladder: **header present** → **JWT valid** → **known user** → **has the Admin.Write scope**, otherwise returns 401/403 status code.

The Middleware Ladder (checked on every request)



The enforcement ladder. The first failed rung decides the status code; later rungs never run.

Hands-On Lab — A Secured MCP Server

The SecurityDemo folder contains five files. Two of them are standalone learning aids; the other three are the running system.

File	What it does
HowJWTWorks.py	A standalone script – no MCP involved. Mints a JWT and validates it so you can watch the encode/decode cycle in isolation. Run this first.
util.py	The stand-in Authorization Server . generate_token() mints an HS256 JWT carrying name, exp and scopes, and writes it to .env;

	validate_token() decodes it.
auth_middleware.py	The enforcement ladder – the part you would still own in production. Starlette middleware that checks header, token, user and scope.
server.py	The MCP Resource Server. Exposes one tool, get_time, and wraps the Streamable HTTP app with the middleware.
client.py	A Streamable HTTP client that reads TOKEN from .env and attaches it as a Bearer header on every request.

Example:

1. git clone <https://github.com/DeccansoftAITeam/MCPDemos.git>
2. cd SecurityDemo
3. python -m venv venv
4. venv\Scripts\activate
5. pip install -r requirements.txt
6. python HowJWTWorks.py
7. Terminal1:
 - a. python util.py
 - b. python server.py
8. Terminal 2:
 - a. python client.py

Code walkthrough

Four ideas carry the whole lab. Read them in this order.

1. util.py – what a token actually asserts

A JWT is only a signed dictionary. Everything the server later decides is already written into this payload:

```

payload = {
    "sub": "1234567890",          # who
    "name": "User Userson",       # identity rung
    "admin": True,
    "iat": datetime.datetime.utcnow(),
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1), # expiry rung
    "scopes": ["Admin.Write", "User.Read"] # authorisation rung
}
encoded_jwt = jwt.encode(payload, secret_key, algorithm="HS256", headers=header)

```

Because the lab signs with a symmetric HS256 secret, the same `secret_key` string must be known to both the minter and the validator – which is why **`auth_middleware.py`** imports **`validate_token`** from `util.py` rather than fetching a public key. In production this is exactly the coupling that asymmetric signing removes.

2. `auth_middleware.py` – the ladder

Each rung returns early, so the first failure decides the status code. Note the split: a missing or malformed header is 401 (you have not identified yourself); a token that is present but unusable is 403 in this lab.

```
USERS = ["User Userson", "Admin Adminson"]
REQUIRED_SCOPE = "Admin.Write"
BEARER_PREFIX = "Bearer "

class CustomHeaderMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        authorization_header = request.headers.get("Authorization")
        if not authorization_header:                # rung 1
            print("-> Missing Authorization header!")
            return Response(status_code=401, content="Unauthorized")

        token = authorization_header[len(BEARER_PREFIX):].strip()
        if not token:                                # rung 2
            return Response(status_code=401,
                            content="Unauthorized - invalid bearer token format")

        decoded_token = validate_token(token)        # rung 3: signature + exp
        if not decoded_token:
            return Response(status_code=403,
                            content="Forbidden - invalid or expired token")

        if not is_user(decoded_token):              # rung 4: known identity
            return Response(status_code=403,
                            content="Forbidden - user does not exist")

        if not has_scope(decoded_token, REQUIRED_SCOPE): # rung 5: RBAC
            return Response(status_code=403,
                            content="Forbidden - insufficient scopes")
```

```

response = await call_next(request)      # only now does MCP run
response.headers["Custom"] = "Example"
return response

```

`call_next` is the gate. Until every rung passes, the MCP request body is never parsed and no tool is ever reached.

3. server.py – where the middleware attaches

Authentication here is transport-level, not per-tool. Instead of calling `mcp.run()`, the lab asks FastMCP for the underlying ASGI application, wraps it, and hands the wrapped app to uvicorn:

```

app = mcp.streamable_http_app()
app.add_middleware(CustomHeaderMiddleware)
uvicorn.run(app, host=mcp.settings.host, port=mcp.settings.port,
             log_level=mcp.settings.log_level.lower())

```

This is the single most important structural point in the module: one middleware protects every tool, resource and prompt the server will ever expose, including ones added later. It also means the check cannot be forgotten on a new tool.

4. client.py – the token on every request

The header is set on the HTTP client, not on a single call, because the specification requires the Authorization header on every HTTP request from client to server – even for requests that are part of the same logical session:

```

token = os.getenv("TOKEN")
http_client = httpx.AsyncClient(headers={"Authorization": f"Bearer {token}"})

async with streamable_http_client(
    url=f"http://localhost:{port}/mcp",
    http_client=http_client) as (read_stream, write_stream, _):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()
        tool_result = await session.call_tool("get_time", {})

```

Try deleting the headers argument: `initialize()` itself fails with 401, before `call_tool` is ever reached.

Verify the auth ladder (tested behaviour)

Request	Server logs	Result
No Authorization header	-> Missing Authorization header!	401 Unauthorized
Valid admin token (default)	User has required scope, proceeding...	get_time returns the time
Valid token without Admin.Write	-> Missing required scope!	403 Forbidden
Expired or tampered token	[FAIL] Token has expired. -> Invalid or expired token!	403 Forbidden
Header present but not “Bearer <token>”	-> Invalid Authorization header format.	401 Unauthorized
Valid token whose name is not in USERS	-> User does not exist!	403 Forbidden

Done when

- util.py writes TOKEN to .env.
- With the default (admin) token, client.py gets get_time.
- No header → 401; token lacking Admin.Write → 403; expired token → 403 (check the server log lines).
- You can place each check in the middleware ladder and say which HTTP status it returns.